SOFTWARE DESIGN SPECIFICATION

Animal Ark Veterinary Society Management System

SENG 31242 - System Design Project

Group - Logic Lords (Group 6)

SE/2022/021

SE/2022/028

SE/2022/033

SE/2022/050

Table of Contents

1. Introduction

1.1 Purpose

This Software Design Specification (SDS) describes the complete technical design of the Veterinary Society Management System (VSMS). It translates the approved Software Requirements Specification (SRS) into a concrete, implementable design ready to hand off for development in SENG 34213. The document covers system architecture, technology justification, MongoDB data models, class specifications, sequence diagrams for all major use cases, UI wireframes, security strategy, and design
constraints.

1.2 Scope

Animal Ark is a web-based platform for a student-led veterinary animal welfare society. The system supports rescue and intake of stray or injured animals, treatment tracking, handover notes between rotating volunteer doctors, supervisor oversight, case closure, emergency alerts, medicine stock control, money donations, blood donor registration, adoption management, volunteer statistics, and a public portal for donors, adopters and animal lovers.

The system supports the following user groups: Admin, Supervisor Doctor, Volunteer Doctor, Donor, Adopter, Blood Donor Owner and Public Visitor. Public visitors can browse published content and submit selected public forms, while restricted operations are protected using authenticated role-based access control.

1.3 References

Reference	**Document**
[SRS]	Software Requirement Specification v1.0 - VSMS
[ADR]	Architectural Decision Records – documents/architecture/decisions/
[UML]	UML 2.x Notation Standard
[OWASP]	OWASP Top 10 Web Application Security Risks (2021)
[Mongoose]	Mongoose ODM Documentation v8.x

2. System Overview

The Animal Ark system is designed as a layered web application. A React single-page application provides role-specific dashboards and a public portal. A Node.js and Express backend exposes REST APIs and coordinates domain services. MongoDB stores core operational data such as animals, cases, treatment records, donations, adoption applications, blood donor registry records and notifications. Redis supports short-lived cache entries, token revocation and notification pub/sub. Socket.IO delivers real-time alerts to dashboards for emergencies, handovers and stock warnings.

The system directly realises the SRS use cases by mapping each functional area to a separate module: Animal/Case, Treatment/Handover, Emergency, Inventory, Donation, Blood Donor, Adoption, Reports and Notification. This separation supports maintainability while keeping deployment simple for the course development timeline.

SRS Area	Design Module	Main Design Artefacts
Animal intake	Animal / Case Module	Animal class, Animal Intake sequence, Animal Case state machine
Treatment timeline and handover	Treatment Module	TreatmentUpdate class, Treatment and Handover sequences
Supervisor oversight	Case Module + Notification Service	Supervisor Review sequence and RBAC rules
Emergency alerts	Emergency Alert Module	EmergencyAlert class, WebSocket interface, Emergency state machine
Medicine stock	Inventory Module	Medicine and StockTransaction classes, stock state machine
Donations	Donation Module	DonationCampaign and Donation classes, PayHere interface
Blood donor registration	Blood Donor Module	BloodDonor class and registry UI
Adoption	Adoption Module	AdoptionApplication class and adoption state machine

SRS Area	Design Module	Main Design Artefacts
Animal intake	Animal / Case Module	Animal class, Animal Intake sequence, Animal Case state machine
Volunteer statistics	Reports Module	VolunteerStatistic class and dashboard sequence
Public portal	Public API + React Portal	Public portal wireframe and navigation flow

3. Architectural Design

3.1 Architecture Pattern

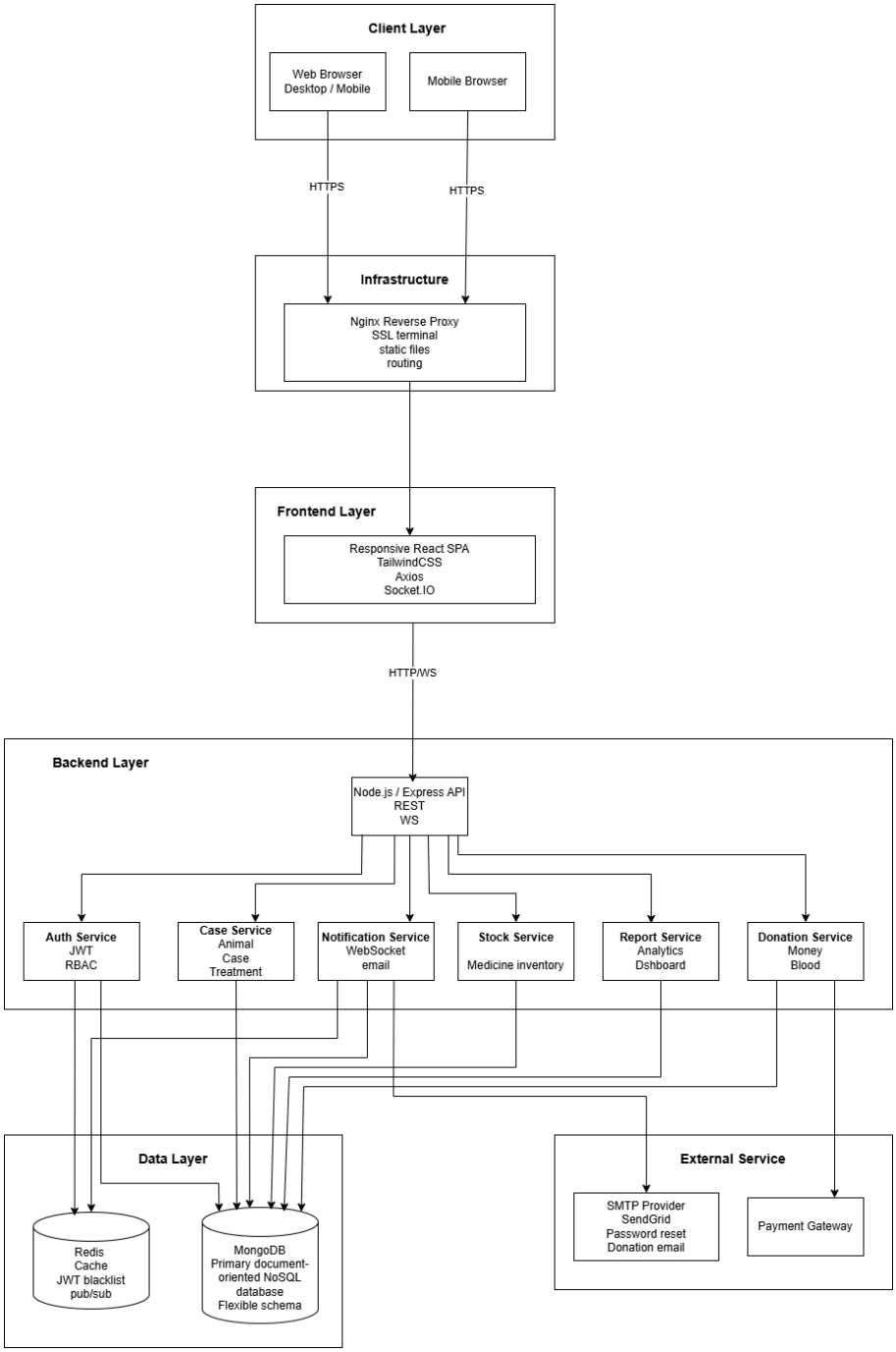
The selected architecture is a Layered N-Tier Architecture with an MVC structure inside the application tier. React components act as the view layer in the browser, Express controllers handle HTTP request routing and validation, service classes implement business rules, and Mongoose models manage persistence. This design was selected instead of microservices because the project team is small and the current scope can be implemented more reliably as a modular monolith. Internal service boundaries are kept clear so that selected modules, such as notification or reporting, can be separated later if required.

Tier	Contents	Reason
Presentation	React SPA, public portal, role-based dashboards	Supports fast UI updates, route guards and reusable components
Gateway	Nginx reverse proxy	Provides HTTPS termination, static file serving and route forwarding
Application	Express API, controllers, services, RBAC middleware, Socket.IO	Centralises business logic and enables maintainable module boundaries
Data	MongoDB, Redis	MongoDB fits flexible animal/treatment records; Redis supports real-time pub/sub and short-lived cache

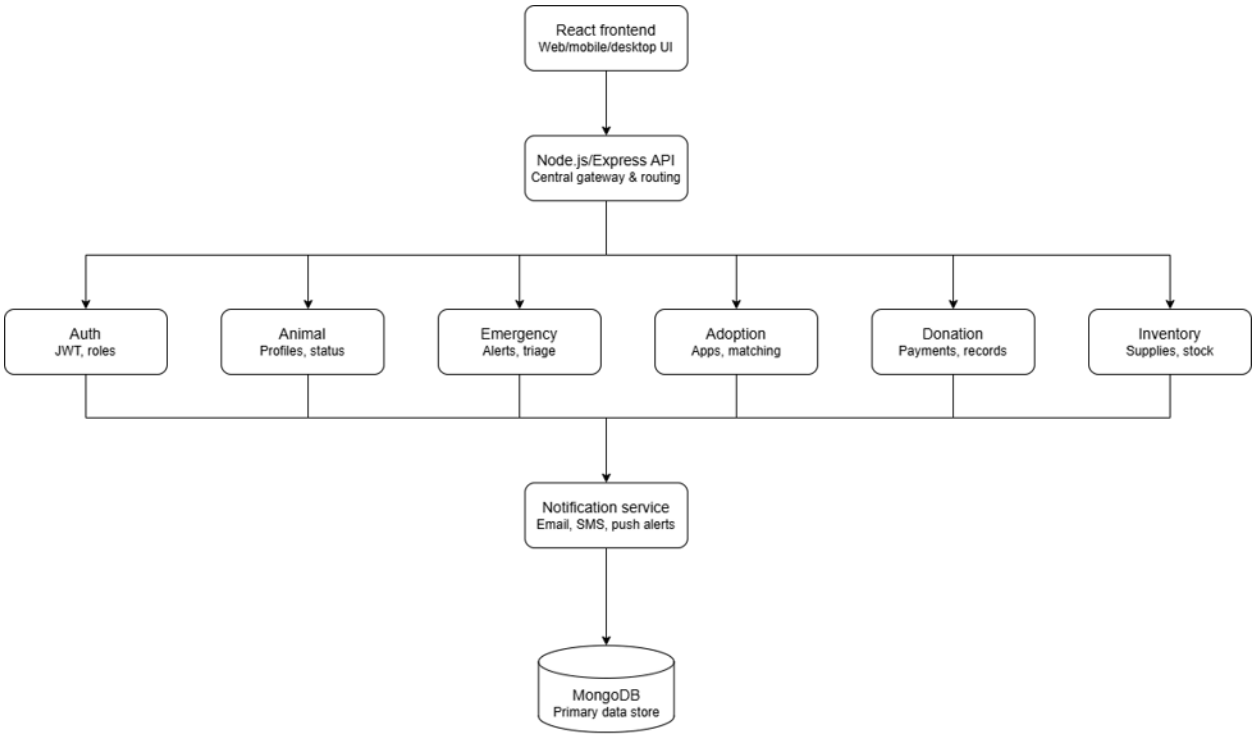
3.2 Architectural Decision Justification

Decision	Rationale	Alternatives Considered	Trade-off Accepted
Layered modular monolith	Simpler development, testing and deployment for a 4-member team while still separating modules	Microservices, serverless functions	Less independent scaling per module, but lower operational complexity
React SPA	Supports dashboards, filtering, timelines and public portal pages with reusable UI components	Server-rendered templates	Requires frontend build pipeline but improves interactive UX
MongoDB with Mongoose	Flexible document structures fit treatment notes, photos, medicine usage arrays and campaign content	Relational SQL database	Requires careful schema validation and indexing
Socket.IO notifications	Real-time emergency and handover communication is a core workflow need	Polling, email only	Adds WebSocket infrastructure but reduces delay for critical alerts
JWT + refresh token	Scalable stateless API authentication with controlled session renewal	Server sessions only	Requires careful token storage and revocation handling

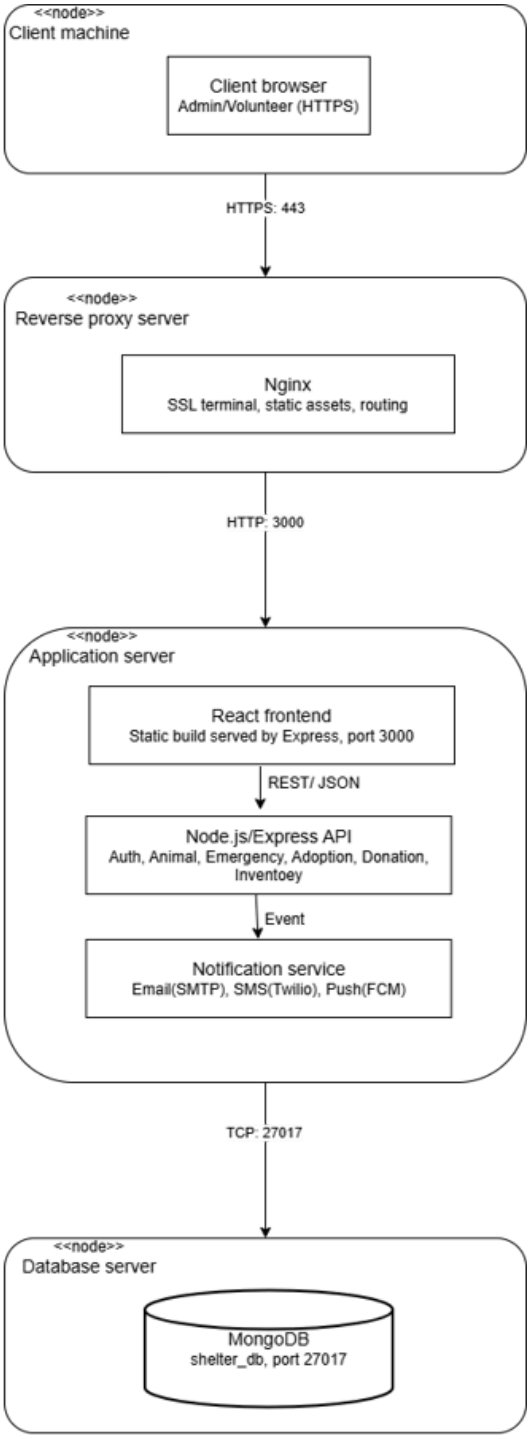
3.3 Architecture Diagram



3.4 Component Diagram



3.4 Deployment Diagram



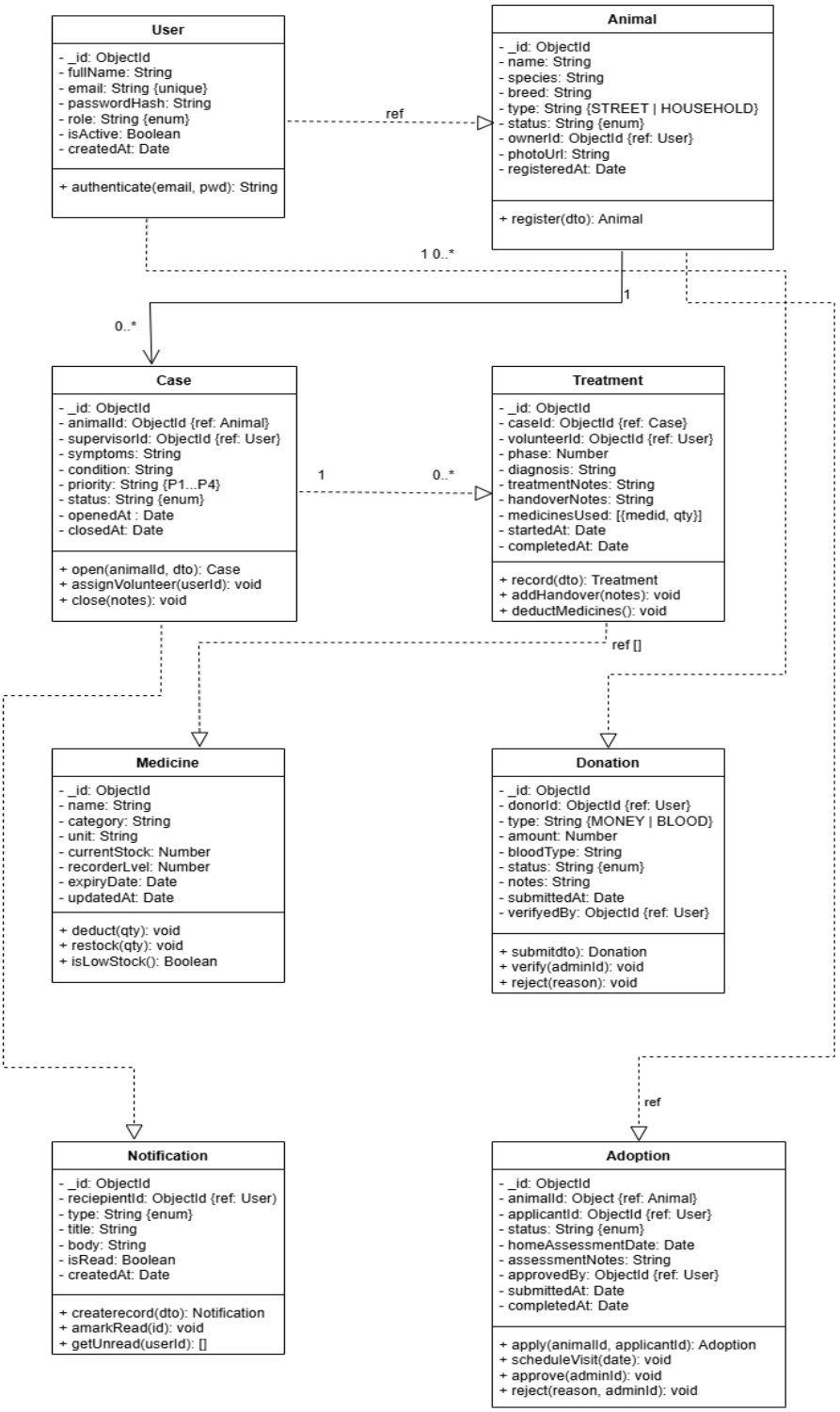
3.5 Technology Stack

Layer	Technology	Version	Justification
Frontend	React	18.x	Component model supports reusable role-specific dashboards and public portal pages
Frontend	React Router	6.x	Client-side routing with protected role-based routes
Frontend	TailwindCSS	3.x	Utility-first styling supports consistent low-overhead UI development
Frontend	Axios	1.x	HTTP interceptors can attach access tokens and handle refresh logic
Backend	Node.js	20 LTS	Non-blocking I/O suits API and real-time notification workloads
Backend	Express.js	4.x	Lightweight routing framework familiar to the team
Backend	Mongoose ODM	8.x	Schema validation, middleware and reference population for MongoDB
Backend	Socket.IO	4.x	Bi-directional real-time events for emergency and handover alerts
Database	MongoDB	7.x	Flexible document store for animal cases, treatment notes and public content
Database	Redis	7.x	Token blacklist, pub/sub and short-lived dashboard KPI cache
Security	bcrypt	5.x	Password hashing with configurable work factor
Security	Joi	17.x	Request validation before controller logic
External	PayHere	Current API	Local payment gateway for Sri Lankan donation payments

External	SendGrid/SMTP	Current API	Password reset, donation acknowledgements and alert emails
Storage	AWS S3 or local file storage	Current	Animal photos with controlled upload and retrieval
DevOps	Docker Compose	v2	Environment parity and simple deployment for development phase
DevOps	GitHub Actions	Current	Automated lint/test/build pipeline during development

4. Detailed Design

4.1 Class Diagram



4.2 Class Specifications

User [users collection]

Represents all system actors via role-based single-collection inheritance.

Attribute	Type	Notes
_id	ObjectId	Auto-generated primary key
fullName	String	Max 100 chars
email	String	Unique index; used for login and notifications
passwordHash	String	bcrypt hash cost 12; never returned in responses
role	String	Enum: ADMIN, SUPERVISOR_DOCTOR, VOLUNTEER_DOCTOR, PET_OWNER
isActive	Boolean	Soft-delete flag
createdAt	Date	Auto-set by Mongoose timestamps
updatedAt	Date	Auto-updated by Mongoose timestamps
Method	**Returns**	**Description**
authenticate(email, pwd)	String	Validates credentials; returns signed JWT pair
resetPassword(token, newPwd)	Boolean	Validates reset token; updates hash
deactivate()	void	Sets isActive=false; adds token to Redis blacklist
toPublicProfile()	Object	Returns safe subset excluding passwordHash

Animal [animals collection]

Represents a street animal or household pet registered in the system.

Attribute	Type	Notes
_id	ObjectId	Auto generated

name	String	Nullable for unnamed street animals
species	String	e.g. Dog, Cat, Bird
breed	String	Optional
estimatedAge	Number	In months; estimated for street animals
gender	String	Enum: MALE, FEMALE, UNKNOWN
type	String	Enum: STREET, HOUSEHOLD
status	String	Enum: REGISTERED, UNDER_TREATMENT, RECOVERED, ADOPTED, DECEASED
photoUrl	String	S3 key or local path; nullable
ownerId	ObjectId	ref: User — null for street animals
registeredAt	Date	Auto-set on creation
Method	**Returns**	**Description**
register(dto)	Animal	Validates and saves new animal document
updateStatus(status)	void	Updates status with updatedAt timestamp
linkOwner(userId)	void	Associates pet owner; validates role=PET_OWNER
getActiveCases()	Case[]	Case.find({animalId, status:{$ne:'CLOSED'}})

Case [cases collection]
Tracks a medical case from opening through closure. One animal may have many sequential cases.

Attribute	Type	Notes
_id	ObjectId	Auto generated
animalId	ObjectId	ref: Animal
supervisorId	ObjectId	ref: User — fixed for case lifetime
symptoms	String	Initial symptom description
condition	String	e.g. Critical, Stable, Recovering

priority	String	Enum: P1_CRITICAL, P2_HIGH, P3_MEDIUM, P4_LOW
status	String	Enum: OPEN, IN_TREATMENT, COMPLETED, CLOSED
openedAt	Date	Auto-set on creation
closedAt	Date	Nullable; set when closed
Method	**Returns**	**Description**
open(animalId, supervisorId, dto)	Case	Creates case; validates animal exists
assignVolunteer(userId)	void	Sets current volunteer; supervisorId is immutable
close(notes)	void	Sets status CLOSED and closedAt=now

Treatment [treatments collection]
Records a single treatment phase administered by one volunteer doctor.

Attribute	Type	Notes
_id	ObjectId	Auto generated
caseId	ObjectId	ref: Case
volunteerId	ObjectId	ref: User
phase	Number	Sequential phase number within a case (1, 2, 3...)
diagnosis	String	Doctor's diagnosis notes
treatmentNotes	String	Procedure performed and observations
handoverNotes	String	Instructions for next volunteer; null if final phase
medicinesUsed	Array	Embedded: [{medId: ObjectId, qty: Number}]
startedAt	Date	When phase began
completedAt	Date	Nullable; set on phase completion

Method	Returns	Description
record(dto)	Treatment	Validates and saves treatment document
addHandover(notes)	void	Sets handoverNotes; triggers next-doctor notification
deductMedicines()	void	Calls StockService for each item in medicinesUsed

Medicine [medicines collection]
Represents a medicine or supply tracked in inventory.

Attribute	Type	Notes
_id	ObjectId	Auto generated
name	String	Generic name, max 150 chars
category	String	e.g. Antibiotic, Analgesic, Vaccine
unit	String	e.g. tablet, ml, vial
currentStock	Number	Current quantity in stock
reorderLevel	Number	Alert threshold
expiryDate	Date	Nullable; earliest batch expiry
updatedAt	Date	Auto updated by Mongoose timestamps
Method	**Returns**	**Description**
deduct(qty)	void	$inc:{currentStock:-qty}; throws if insufficient
restock(qty)	void	$inc:{currentStock:+qty}; logs restock event
isLowStock()	Boolean	Returns currentStock <= reorderLevel

Donation [donations collection]
Records a money or blood donation from a registered donor.

Attribute	Type	Notes
_id	ObjectId	Auto generated
donorId	ObjectId	ref: User

type	String	Enum: MONEY, BLOOD
amount	Number	Nullable for blood donations
bloodType	String	Nullable for money donations
status	String	Enum: PENDING, VERIFIED, REJECTED
notes	String	Optional donor notes
submittedAt	Date	Auto-set on submission
verifiedAt	Date	Nullable; set by admin
verifiedBy	ObjectId	ref: User (admin); nullable
Method	**Returns**	**Description**
submit(dto)	Donation	Validates and creates donation document
verify(adminId)	void	Sets VERIFIED; sends acknowledgement email
reject(reason)	void	Sets REJECTED; notifies donor

Notification [notifications collection]
Persisted notification record, also emitted in real time via Socket.IO.

Attribute	**Type**	**Notes**
_id	ObjectId	Auto generated
recipientId	ObjectId	ref: User
type	String	Enum: TREATMENT_UPDATE, EMERGENCY, ASSIGNMENT, STOCK_LOW
title	String	Max 80 chars
body	String	Full notification body
isRead	Boolean	Default false
createdAt	Date	Auto-set
Method	**Returns**	**Description**

create(dto)	Notification	Saves document; emits via Socket.IO to user:<id> room
markRead(id)	void	Sets isRead=true
getUnread(userId)	Notification[]	Returns all unread for a user

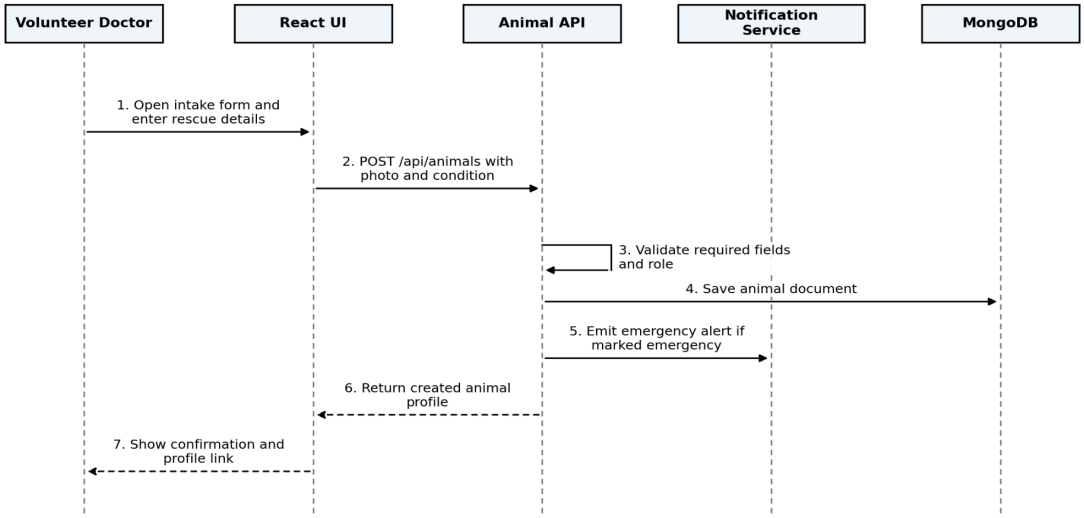
Adoption [adoptions collection]
Tracks the full lifecycle of an animal adoption application.

Attribute	**Type**	**Notes**
_id	ObjectId	Auto generated
animalId	ObjectId	ref: Animal
applicantId	ObjectId	ref: User
status	String	Enum: PENDING, UNDER_REVIEW, APPROVED, REJECTED, COMPLETED
homeAssessmentDate	Date	Nullable until scheduled
assessmentNotes	String	Admin field notes from home visit
approvedBy	ObjectId	ref: User (admin); nullable
submittedAt	Date	Auto-set on application
completedAt	Date	Nullable; set when animal handed over
Method	**Returns**	**Description**
apply(animalId, applicantId)	Adoption	Creates PENDING application
apply(animalId, applicantId)	void	Sets homeAssessmentDate; notifies applicant
approve(adminId)	void	Sets APPROVED; Animal.findByIdAndUpdate status=ADOPTED
reject(reason, adminId)	void	Sets REJECTED; notifies applicant with reason

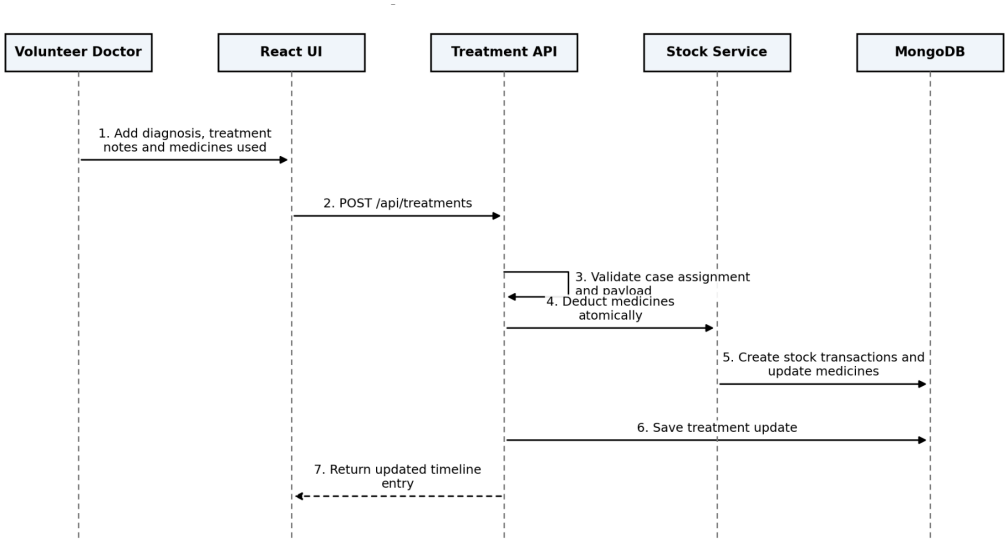
4.3 Sequence Diagrams

One sequence diagram is produced per major use case. Source files (draw.io) and SVG exports are in documents/sds/diagrams/sequence/. The message flows below are the authoritative textual specification.

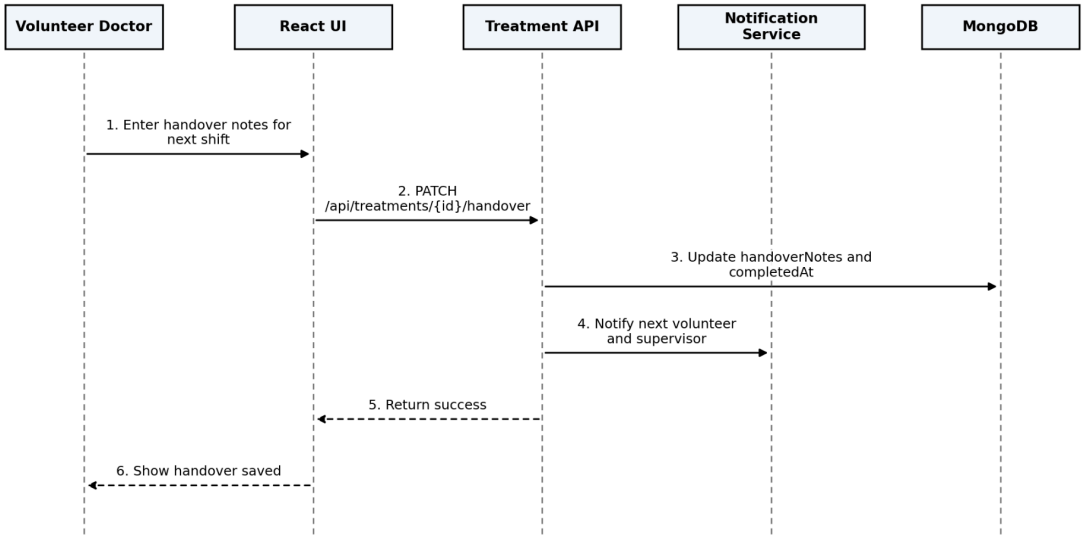
SD-01 Animal Intake and Profile Creation [UC-01]



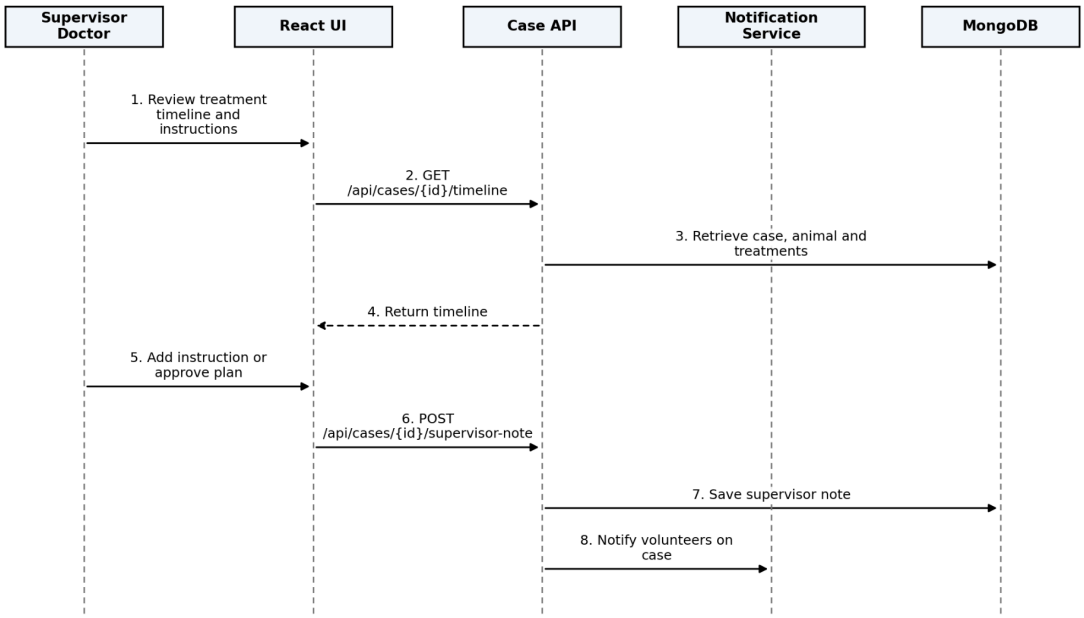
SD-02 Treatment Timeline Management [UC-02]



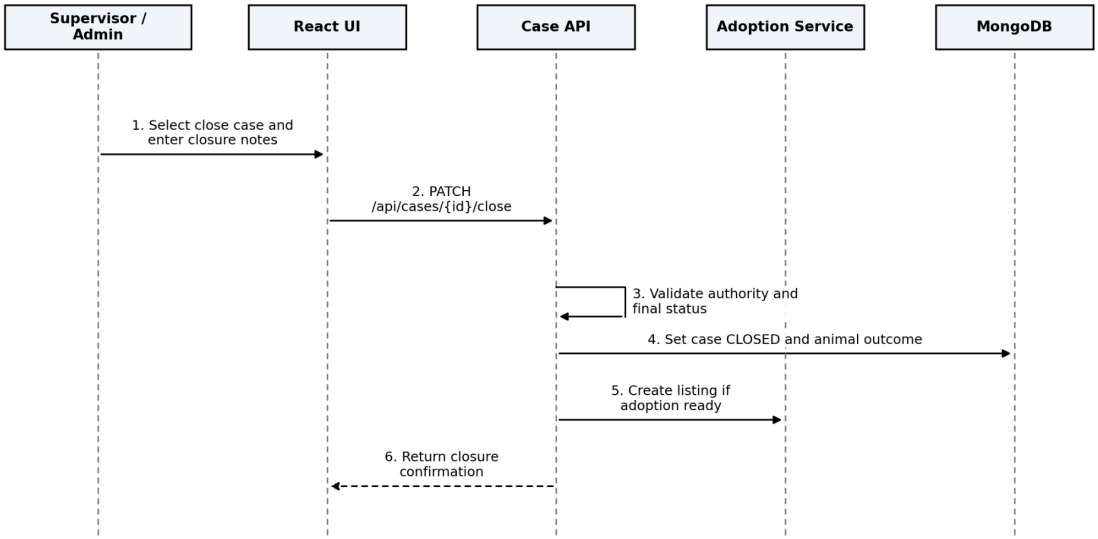
SD-03 Handover Notes Management [UC-03]



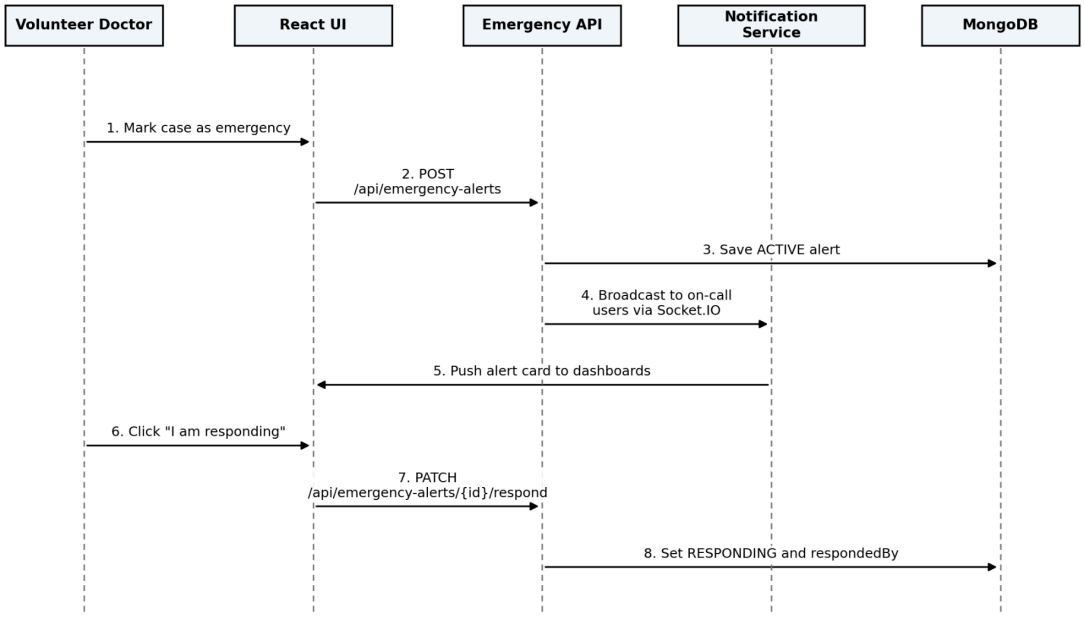
SD-04 Supervisor Oversight and Review [UC-04]



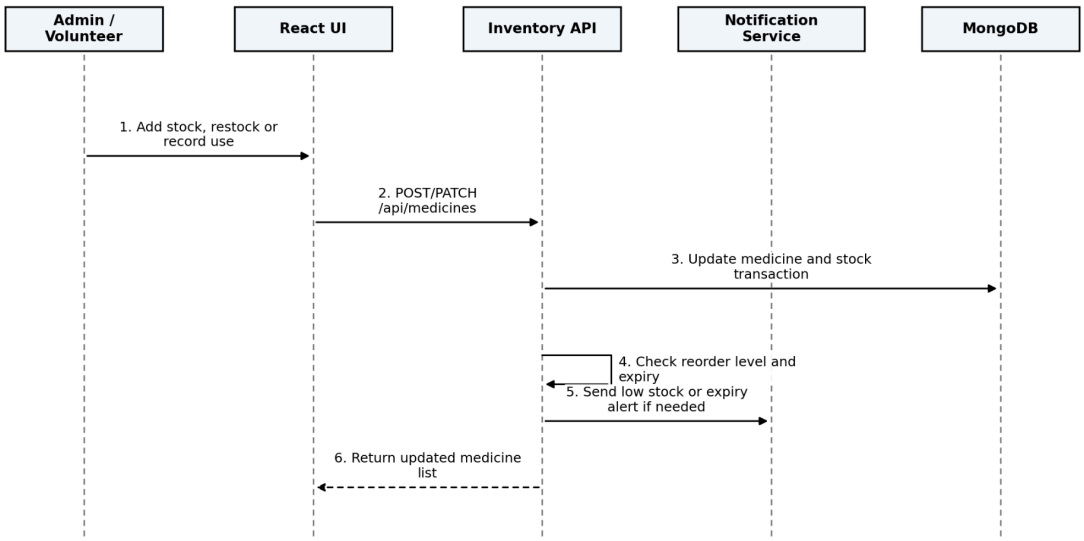
SD-05 Case Closure [UC-05]



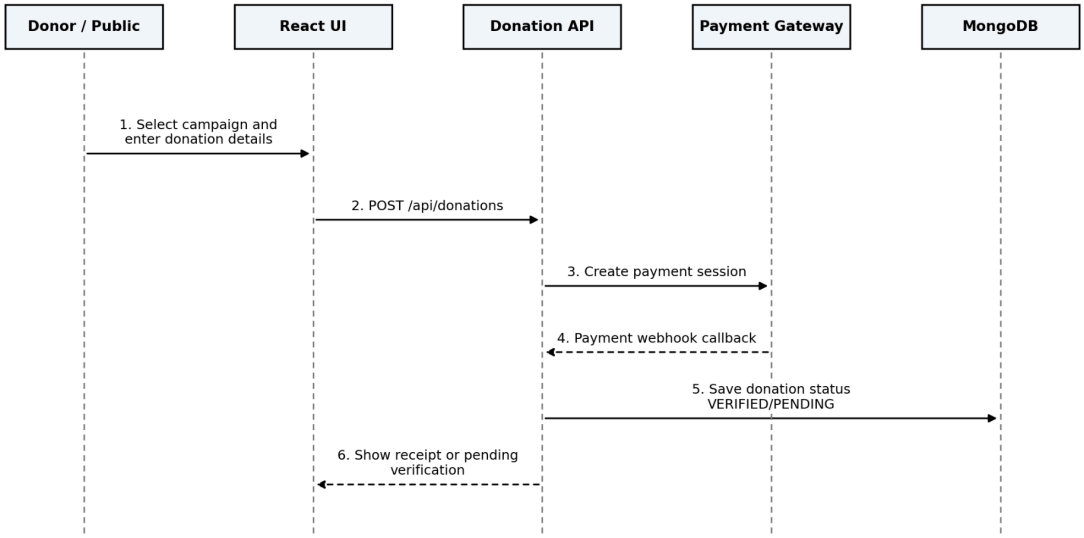
SD- 06 Emergency Alert Management [UC-06]



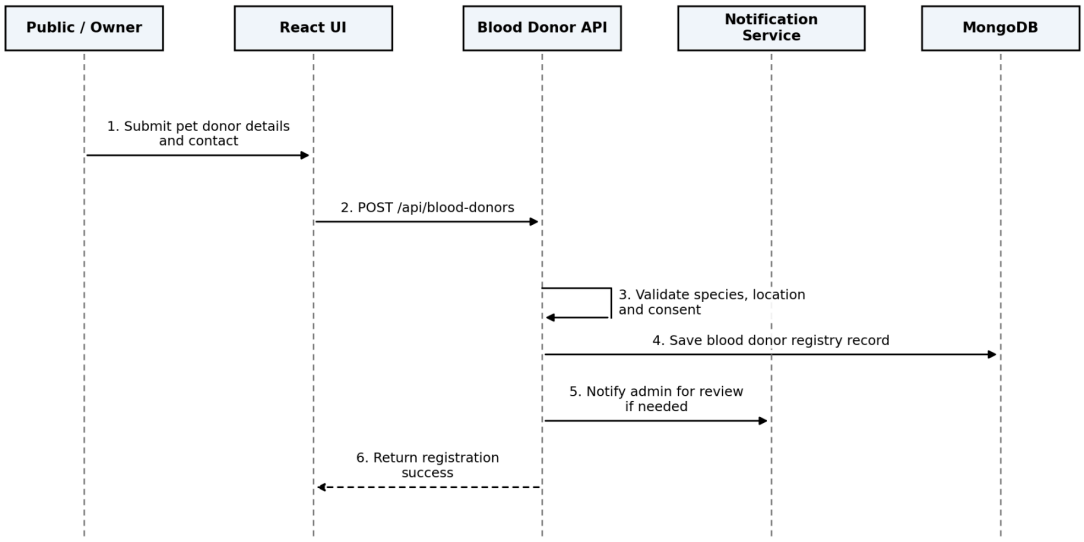
SD-07 Medicine Stock Management [UC-07]



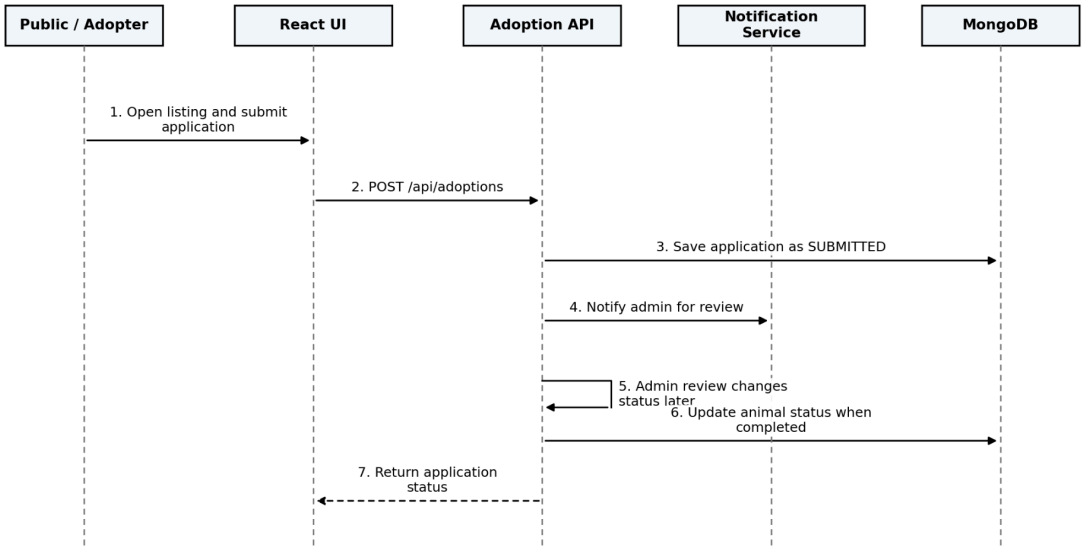
SD-08 Donation Campaign and Donation Record Management [UC-08]



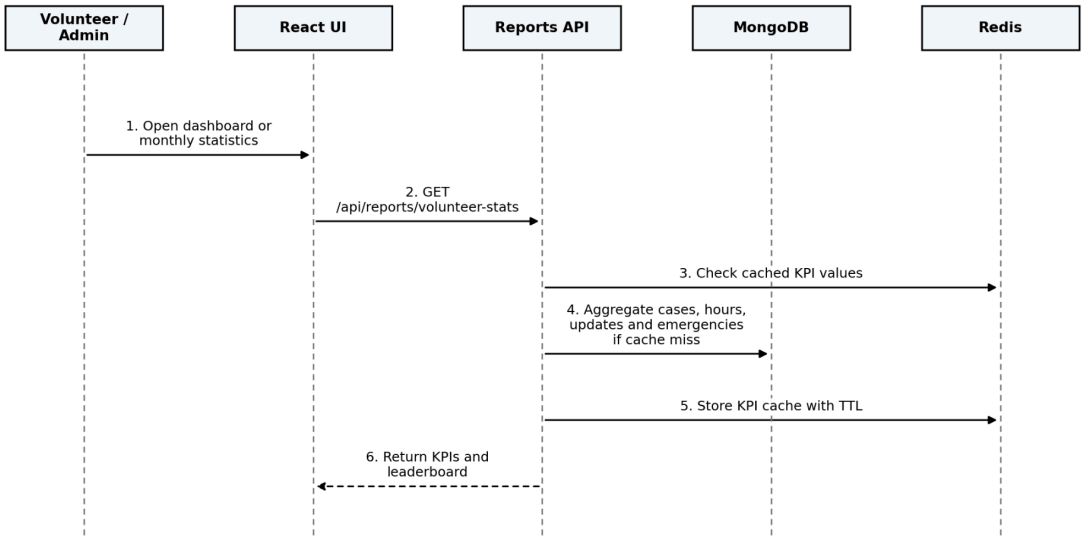
SD-09 Blood Donor Registration and Search [UC-09]



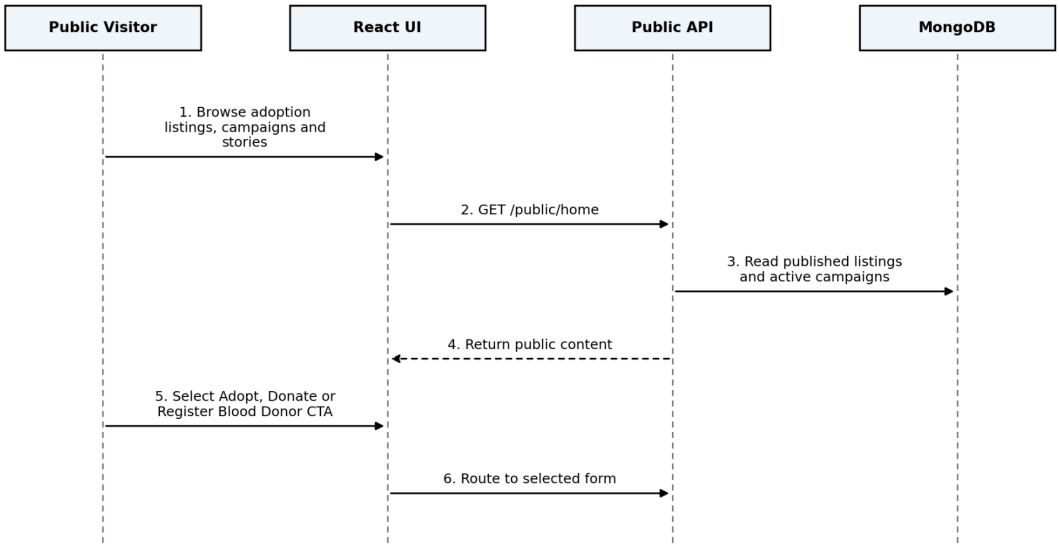
SD-10 Adoption Flow Management [UC-10]



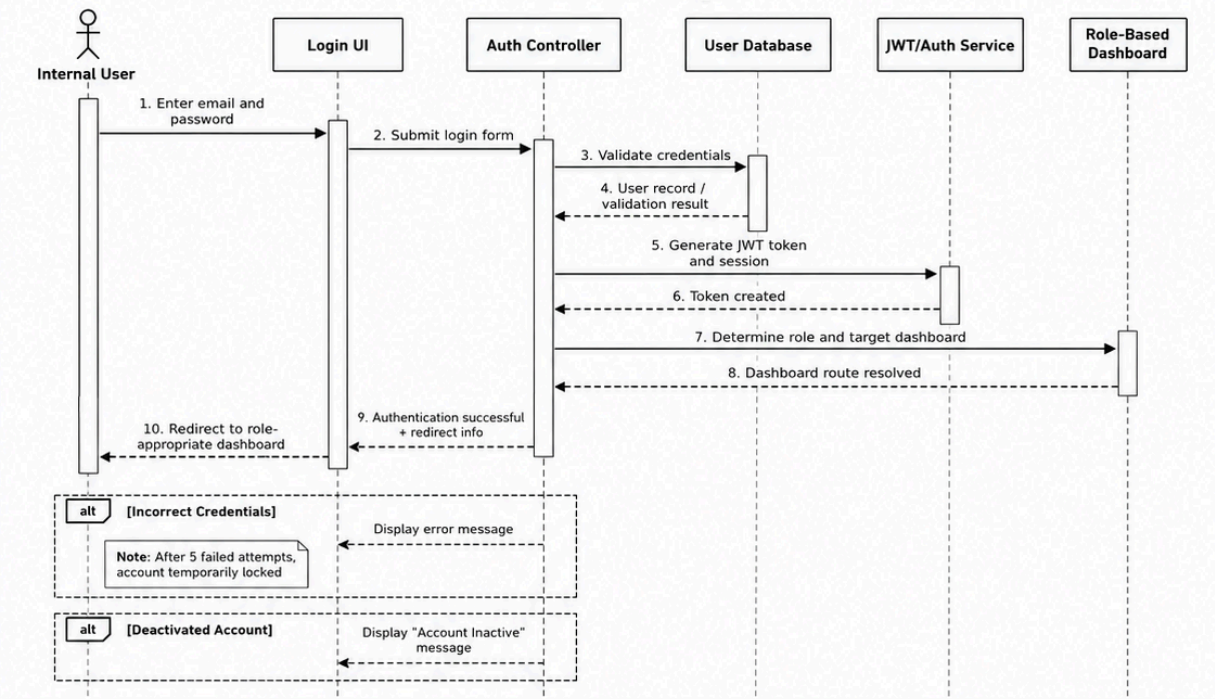
SD-11 Volunteer Statistics Tracking [UC-11]



SD-12 Public Portal Access [UC-12]

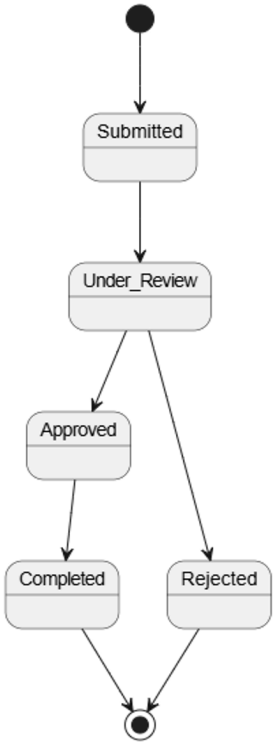


SD-13 User Authentication and Role Management [UC-13]

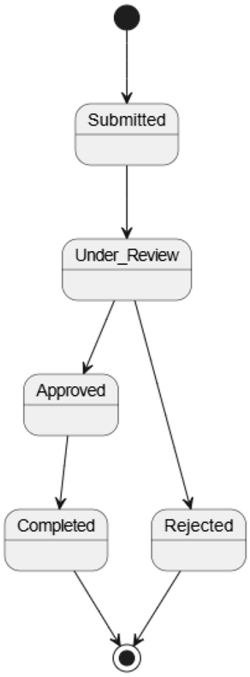


4.4 State Machine Diagrams

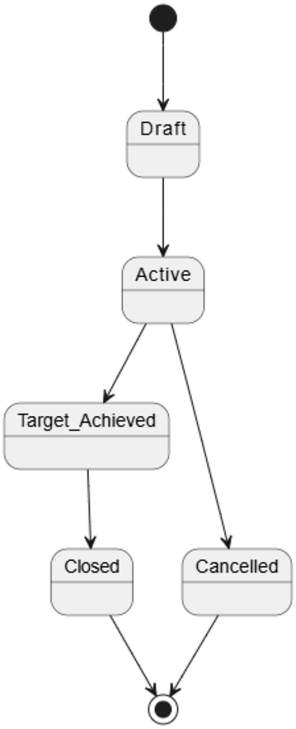
1. Animal Intake



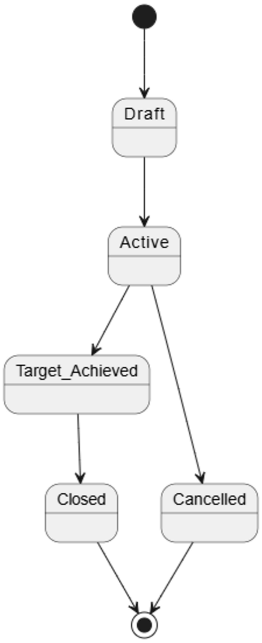
2. Adoption Request



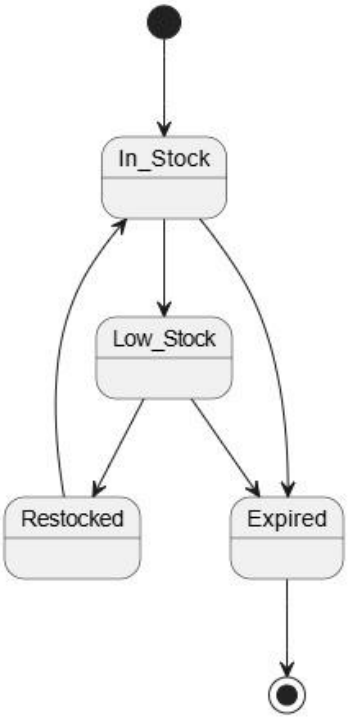
3. Emergency Alert



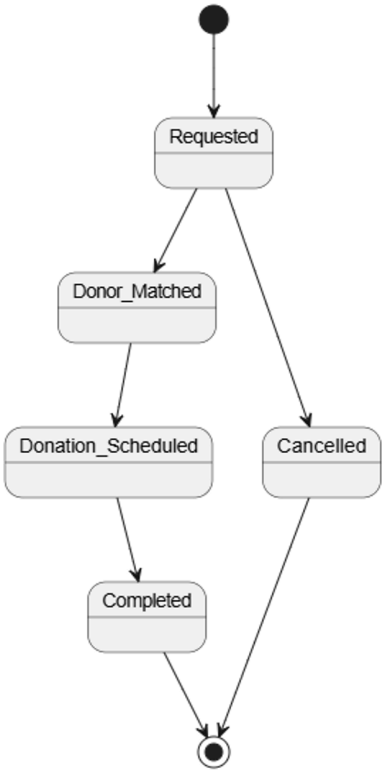
4. Donation Campaign



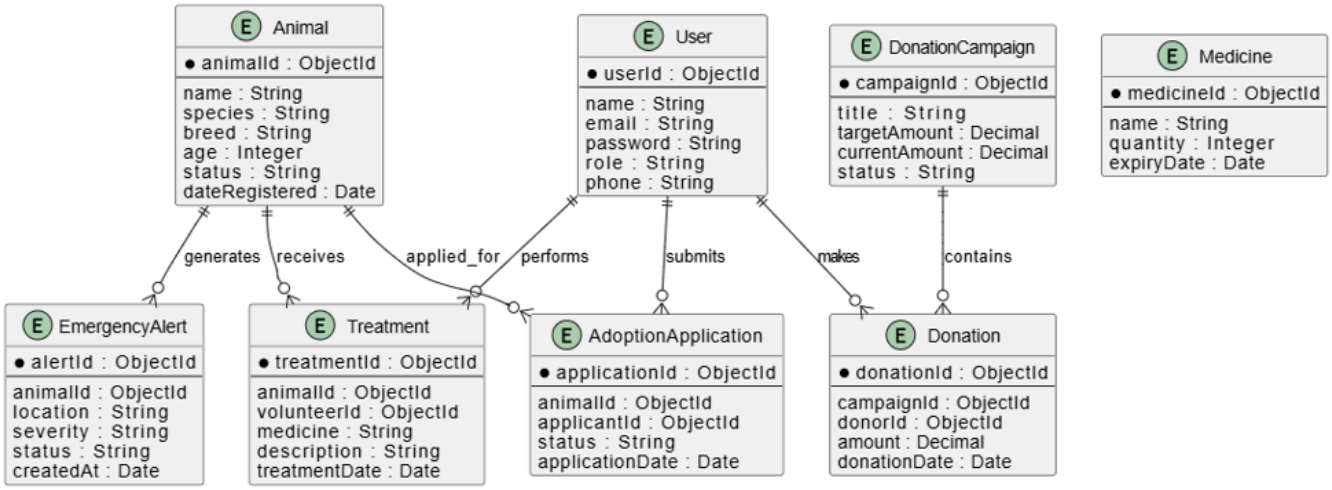
5. Medicine Stock



6. Blood Donation Request



4.5 ER Diagram



4.6 MongoDB Collections

The system uses 8 MongoDB collections. All document IDs are ObjectId. References between collections use ObjectId fields with Mongoose ref strings. No join tables are needed, embedded sub-documents handle nested data (e.g. medicinesUsed in Treatment).

Collection	Purpose
users	All actors : Admin, Supervisor Doctor, Volunteer Doctor, Pet Owner, Donor
animals	Registered street animals and household pets
cases	Medical cases linked to animals
treatments	Treatment phases within a case
medicines	Medicine inventory with stock levels
donations	Money and blood donation records
notifications	Persisted notification records per user
adoptions	Adoption applications and their lifecycle

5. User Interface Design

5.1 Wireframe Descriptions

All wireframe source files are in documents/sds/wireframes/. The specifications below are the authoritative textual record for each screen, derived from the approved Figma prototype.

WF-01 Log In (All users)

- Animal Arc paw logo and "Animal Arc" heading centered at top of card
- "Veterinary Society Management" subtitle in muted text below heading
- EMAIL labelled text input field
- PASSWORD labelled masked input field with "Forgot Password?" link inline on the right
- Full-width "Sign in" primary button
- "Don't have an account? Sign up" link below button navigates to WF-02
- Role is determined from JWT payload after sign-in; no role selector shown on this screen
- Inline error banner appears above email field on failed credentials

WF-02 Sign Up (All new users)

- Animal Arc paw logo and "Animal Arc" heading centered at top of card
- "Veterinary Society Management" subtitle in muted text below heading
- NAME labelled text input field
- EMAIL labelled text input field with format validation
- PASSWORD labelled masked input field
- CONFIRM PASSWORD labelled masked input field with inline match validation
- ROLE labelled dropdown or segmented selector
- Full width "Sign up" primary button
- "Already have an account? Log in" link below button navigates back to WF-01
- Successful sign-up redirects to Dashboard (WF-03) after account creation

WF-03 Dashboard (Volunteer / Supervisor / Admin)

- Top navigation bar: Animal Arc logo on the left, user avatar on the right
- Three KPI stat cards in a row: Active Rescued, My Assigned, Emergencies
- MY ACTIVE CASES section: vertical list of case cards, each showing animal initial avatar (coloured circle), animal name and case ID, condition description,

and a colour-coded status badge (Treatment → amber, Stable → green, Monitored → blue)
- Blue navigation arrows on left and right sides of the case list for pagination
- Navigation flow connections visible to Animal Profile, Animals list, Sign Up, Adoption, and Emergency Alert screens

WF-04 Animal Profile (Volunteer / Supervisor)

- Back arrow and breadcrumb: "Case #038 Dog"
- "Active treatment" status badge and "+ Add update" button in page header
- Left column Animal details card:
    - Real animal photo displayed (dog photo visible in wireframe)
    - Metadata table: Species, Breed, Est. age, Intake date, found at, Condition tag
- Right column Care team panel:
    - Supervisor section: avatar initials (DW), doctor name, "Assigned supervisor All updates" label
    - Volunteers on case: row of three circular avatar bubbles with initials (JW, KP, NS)
    - Current medicines section: medicine name, dose, and frequency per active medicine
- TREATMENT TIMELINE section (full width below):
    - Vertical chronological list; each entry shows date and time, volunteer name, note text, and an entry-type badge
    - Entry badge types: Improving → green, Handover note → blue, Supervisor instruction → purple, Intake → gray
    - Entries with unread status shown with an open circle indicator on the left

WF-05 Animals List (Volunteer / Supervisor / Admin)

- Top bar: search input and Filter button on the left; "+ Register animal" primary button top-right
- Left filter sidebar: FILTER BY STATUS pills (All, Emergency, Active, Recovered, Adopted with counts); FILTER BY TYPE pills (Dog, Cat, Other with counts)
- Main content: responsive image grid of animal cards (3 columns visible)
- Each animal card: actual animal photo, colour-coded status badge overlaid on photo, animal name and case ID, short condition note, "View case" button

- Status badge colours: Emergency → red, Treatment → amber, Stable → green, monitored → blue, Recovered → dark gray, Adoption ready → teal
- Clicking "View case" navigates to WF-04 Animal Profile
- "+ Register animal" button navigates to WF-06 Animal Intake Form

WF-06 Animal Intake Form (Volunteer)

- Page heading "Register new animal" with back arrow
- Two-column form layout:
    - Left column: SPECIES * dropdown, BREED text input, ESTIMATED AGE text input, SEX dropdown, FOUND LOCATION * text input, RESCUE DATE & TIME * datetime picker
    - Right column: IMAGE upload area (drag-and-drop or file picker with preview), CONDITION multi-line textarea, ASSIGNED TO volunteer picker dropdown
- "Is this an emergency?" toggle at bottom: Yes (red "alert on-call team") / No
- CANCEL button and "Submit" primary button at bottom-right
- Selecting Yes on the emergency toggle fires a real-time alert to the on-call supervisor
- From validates all required fields (* marked) before submission

WF-07 Emergency Alert (Volunteer / Supervisor)

- Page heading "Emergencies" with notification bell icon
- Active emergency alert cards (red bordered):
    - Card 1: "Road accident Dog (Case #042)" ACTIVE badge, time-ago label, description text, location, admitted-by name, notified count; "I'm responding," button and "View case" button
    - Card 2: "Critical Cat (Case #039)" ACTIVE badge, condition deteriorated note, supervisor approval required label; "Review & approve" and "View case" buttons
- RESOLVED (LAST 7 DAYS) section below active cards:
    - Each resolved item: green tick icon, case name, resolution date, responder name, response time in minutes
- Active cards refresh automatically via WebSocket; new alerts push to top of list
- "I'm responding" updates case assignment and collapse the card for other volunteers
- "Review & approve" opens an inline approval modal for supervisors

WF-08 Medicine Stock (Volunteer / Admin)

- Page heading "Medicine stock management" with Volunteer / Admin role badge
- "+ Add stock" primary button top-right
- Four KPI cards in a row: Total items (34, white), Low stock (3, red), Expiring soon (2, amber), Used this week (12, teal)
- Amber warning banner below KPIs: medicine name, days until expiry, tablets remaining, recommended action text
- "Search medicines…" text input and Filter button
- Medicine list rows: pill icon, medicine name, category and expiry date, color-coded stock-level badge, quantity text, "Use" button
- Thin color-coded progress bar beneath each medicine row showing stock level relative to maximum capacity
- Badge colors: Green → sufficient, Amber → low, Red → critical / out of stock
- "Use" button opens a quantity entry modal; on confirm, stock decremented and a treatment log entry created automatically

WF-09 Blood Donation Management (Volunteer / Admin / Public)

- Page heading "Blood Donation Management" with "+ Register donor" primary button top-right
- Search input "Search by species, blood type, location…" with Species and Location dropdown filters
- BLOOD DONOR REGISTRY list:
    - Each row: blood-drop icon, pet name with breed, age and sex, blood type, owner name and contact phone, location, Availability badge (Available → green, Busy until [month] → amber), "Contact" button
- REGISTER A BLOOD DONOR inline form section below the registry:
    - PET NAME, SPECIES, BLOOD TYPE (if known), OWNER CONTACT, LOCATION inputs
    - "Register donor" submit button
- Registered donors are immediately searchable and contactable for emergency matching

WF-10 Donation Management Volunteer/Admin view (Volunteer / Admin)

- Page heading "Donation & Campaigns" with role badge
- Three KPI cards: LKR raised this month, Active campaigns count, Pending verification (red)

- ACTIVE CAMPAIGNS section: each campaign card shows name, description, LKR raised, goal amount, green progress bar, "View donors" button and "Record donation" button
- RECENT DONATIONS list donor name, donation amount, fund name, date, verified (green) or Pending (amber) status badge
- All data accessible to Volunteer role for recording and viewing purposes

WF-10b Donation Management Admin Panel view (Admin only)

- Identical layout to WF-10 with additional admin-only controls:
    - "+ New campaign" primary button top-right
    - "Manage campaign" edit button on each campaign card
    - Pending verification amount highlighted in red KPI card
    - "Verify" and "Reject" action buttons on each recent donation row
    - Full donor list export accessible via "View donors" modal

WF-11 Adoption (Admin / Public)

- Page heading "Adoption" with "+ Create listing" primary button top-right
- Four KPI cards: Listed for adoption, Applications, pending review, Adopted (total)
- AVAILABLE FOR ADOPTION left column:
    - Each listing card: paw placeholder photo, Ready badge, case ID, species and age, trait tags (e.g. Friendly / Vaccinated / Good with people), "Edit listing" button and "{n} apps" application count button
- PENDING APPLICATIONS right column:
    - Each row: applicant avatar initials, applicant full name, "Applying for Case #0XX" label, submission date, "Review" button
- "Review" button opens applicant detail modal with home type, pet experience, declaration, Approve and Reject action buttons
- Approving sets Animal.status = ADOPTED in database and notifies applicant

WF-12 MyStats / Volunteer Statistics (Volunteer / Admin)

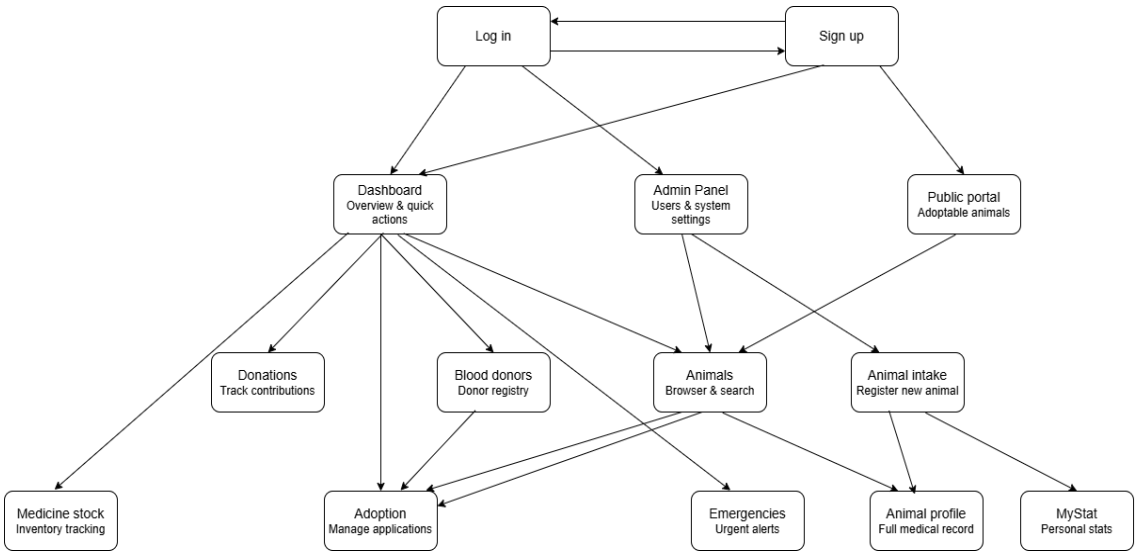
- Page heading "Volunteer Statistics" with month selector dropdown top-right (Month: May 2026)
- Left panel best volunteer of the month highlighted card:
    - Name, Cases count (6), Hours total (62h)
- Right panel MY STATS for the selected month: four tiles:

    - Cases (3), Hours (47h), Updates (12), Emergencies (2)
- LEADERBOARD section (full width below):
    - Ranked rows: rank number with color indicator, avatar initials circle, volunteer name, cases + hours + updates summary
    - Rank #1 has a trophy icon
    - Current logged-in user's row highlighted with blue border

WF-13 Public Portal (Donors / Adopters / Public no login required)

- Shield/paw logo in top navigation bar
- Top navigation links: Home, Adopt, Donate, Blood donors, Stories, Contact
- Hero section: "EVERY RESCUED LIFE COUNTS" headline, society description subtitle, three primary CTA buttons "Adopt an animal", "Make a donation", "Register blood donor"
- SUPPORT A CAMPAIGN section (right of hero):
    - Campaign name, description, LKR raised and goal, progress bar, "Donate now" button
- ANIMALS AVAILABLE FOR ADOPTION grid (below hero):
    - Each card: actual animal photo, species and age, two trait tags, "Apply" button
- SUCCESS STORIES section: animal name and recovery headline, timeline summary, outcome (e.g. adopted)
- "Apply" navigates to adoption application form no account required
- "Donate now" navigates to payment form with PayHere integration
- "Register blood donor" navigates to WF-09 Blood Donor Registry

5.2 Navigation Flow



https://www.figma.com/design/3aRVb2OpZpIykqfoJUlqST/Healthcare-Dashboard-UI--Community-?node-id=0-1&t=ip9aLNfqyGt243dc-1

6. Interface Design

6.1 External Interfaces

External Interface	Purpose	Data Exchanged	Design Notes
PayHere Payment Gateway	Process money donations	Donation amount, campaign ID, donor reference, payment status webhook	Webhook signature is verified before setting donation status to VERIFIED
SendGrid / SMTP	Transactional email	Password reset links, donation receipts, adoption updates, alert summaries	Email sending is wrapped in NotificationService with retry logging
AWS S3 or Local File Storage	Animal and case photo storage	Image file, object key/path, metadata	Uploads are restricted by MIME type and size; stored path is saved in MongoDB
Socket.IO WebSocket	Real-time notifications	Emergency alerts, handover updates,	Users join role/user rooms after JWT authentication

		low-stock alerts, assignment changes	
Browser / React Client	Human interaction with system	REST requests, form data, JSON responses, images	Protected routes redirect unauthorised users to login or public portal

6.2 Internal Module Interfaces

Provider Module	Consumer Module	Interface / Operation	Purpose
Auth Module	All API modules	requireAuth(), requireRole(), requireOwnership()	Verify identity and authorisation before business logic
Animal / Case Module	Treatment Module	getCaseById(), assertAssignedVolunteer()	Ensure treatment updates belong to valid assigned cases
Inventory Module	Treatment Module	deductMedicines(caseId, items)	Deduct stock atomically when medicines are used
Notification Service	All domain services	notifyUser(), notifyRole(), emitSocketEvent()	Send in-app, WebSocket and email notifications
Donation Module	Payment Adapter	createPayment(), verifyWebhook()	Coordinate donation payment workflow
Adoption Module	Animal Module	markAdopted(animalId)	Update animal status after completed adoption
Reports Module	Case, Treatment, Emergency modules	aggregateVolunteerStats()	Calculate monthly KPIs and leaderboard

6.3 REST API Interface Summary

Method	Endpoint	Allowed Roles	Purpose
POST	/api/auth/login	Public	Authenticate user and return access token plus refresh cookie
POST	/api/auth/register	Public/Admin depending role	Create user account with allowed role
POST	/api/animals	VOLUNTEER_DOCTOR, ADMIN	Register rescued or household animal profile
GET	/api/animals	VOLUNTEER_DOCTOR, SUPERVISOR_DOCTOR, ADMIN	Search and filter animals
POST	/api/cases	SUPERVISOR_DOCTOR, ADMIN	Open treatment case for animal
PATCH	/api/cases/{id}/assign-volunteer	SUPERVISOR_DOCTOR, ADMIN	Assign current volunteer doctor
PATCH	/api/cases/{id}/close	SUPERVISOR_DOCTOR, ADMIN	Close case with final outcome
POST	/api/treatments	VOLUNTEER_DOCTOR	Record treatment timeline entry
PATCH	/api/treatments/{id}/handover	VOLUNTEER_DOCTOR	Add handover notes for next shift
POST	/api/emergency-alerts	VOLUNTEER_DOCTOR, SUPERVISOR_DOCTOR	Create emergency alert for case
PATCH	/api/emergency-alerts/{id}/respond	VOLUNTEER_DOCTOR, SUPERVISOR_DOCTOR	Mark user as responding
GET/POST /PATCH	/api/medicines	VOLUNTEER_DOCTOR, ADMIN	View and manage medicine stock according to role
POST	/api/donations	Public, DONOR	Submit donation or start payment
PATCH	/api/donations/{id}/verify	ADMIN	Verify or reject donation

POST	/api/blood-donors	Public, ADMIN	Register blood donor animal and owner contact
POST	/api/adoptions	Public, ADOPTER	Submit adoption application
PATCH	/api/adoptions/{id}/review	ADMIN	Approve, reject or schedule assessment
GET	/api/reports/dashboard	VOLUNTEER_DOCTOR, SUPERVISOR_DOCTOR, ADMIN	Retrieve role-specific dashboard KPIs
GET	/public/home	Public	Retrieve adoption listings, campaigns and success stories

7. Security Design

7.1 Authentication

Mechanism	**Detail**
Password hashing	bcrypt with cost factor 12. Never stored or logged in plain text
Access token	JWT signed with HS256, expires in 15 minutes
Refresh token	Opaque token, 7-day expiry, stored in HTTP-only cookie (not localStorage)
Token revocation	Revoked tokens stored in Redis with TTL matching remaining validity
Password reset	Time-limited (30 min) single-use token sent via email

7.2 Authorization – Role Based Access Control (RBAC)

Role-Based Access Control (RBAC) is applied to every protected route. The middleware verifies the JWT, checks the token blacklist, attaches the user identity and role to the request, and blocks roles that are not permitted. Ownership checks are applied where required, such as allowing a user to view only their own adoption application unless they are an admin.

Role	Main Permissions
ADMIN	Manage users, verify donations, manage campaigns, review adoptions, view reports, close or correct records
SUPERVISOR_DOCTOR	Open/close cases, assign volunteers, review treatment plans, approve emergency actions, view medicine stock
VOLUNTEER_DOCTOR	Register animals, update treatment timeline, add handover notes, respond to emergencies, use medicine stock
DONOR	Submit donations and view own receipts if registered
ADOPTER	Submit and track own adoption applications if registered
BLOOD_DONOR_OWNER	Register and update own blood donor animal details if registered
PUBLIC	Browse public portal, view adoption listings, submit guest donation/adoption/blood donor forms as allowed

7.3 Input Validation

- All request bodies validated with Joi schemas before any controller logic runs
- Mongoose schema validation as a second layer (type, required, enum, minlength)
- File uploads: MIME type whitelist (image/jpeg, image/png), size limit 5 MB, filename sanitised
- No raw query string interpolation all MongoDB queries use parameterised Mongoose calls

7.4 OWASP Top 10 Mitigations

OWASP Risk	Mitigation
Broken Access Control	RBAC middleware on every protected route, service-level ownership checks and audit logs for admin actions
Cryptographic Failures	HTTPS, bcrypt password hashing, environment-managed secrets and no plaintext credential storage
Injection	Joi validation, Mongoose parameterised operations and no eval/dynamic query string construction
Insecure Design	Documented architecture decisions, role separation and threat-aware workflow design for payments and public forms
Security Misconfiguration	Production CORS allow-list, secure cookies, environment-based config and container hardening
Vulnerable Components	Dependency audit in CI and periodic updates during development

Identification and Authentication Failures	Short token lifetime, refresh token revocation, rate limiting and generic password reset responses
Software and Data Integrity Failures	Webhook signature verification and GitHub pull request review process
Logging and Monitoring Failures	Structured logs for auth events, role changes, payment webhooks, case closure and stock deductions
Server-Side Request Forgery	No unrestricted URL fetching from user input; external integrations use fixed configured endpoints

8. Non-Functional Design Decisions

This section maps the non-functional requirements from the SRS to concrete design decisions. It is included as a required SDS section and should be kept consistent with the final SRS NFR identifiers.

NFR Category	**Design Decision**	**How the Design Satisfies It**
Performance	Redis cache for dashboard KPIs and indexed MongoDB queries	Frequently accessed dashboard counts are cached with short TTL; list pages use pagination and indexes.
Scalability	Layered modular monolith with stateless API and Redis-backed Socket.IO	The API can be horizontally scaled later because session state is not stored in memory.
Maintainability	Module-based service layer and ADRs	Each domain module owns its controller, service and model files; major decisions are recorded with rationale and trade-offs.
Reliability	Atomic treatment-stock transaction and Docker health checks	Medicine deduction and treatment entry are committed together; containers can auto-restart on failure.
Security	RBAC, JWT, bcrypt, HTTPS and validation	Access is restricted by role, credentials are protected, and input is validated on client and server.

Usability	Role-specific dashboards, colour-coded badges and timeline UI	Users see only relevant work and can understand case status quickly.
Availability	VPS deployment with backups and restart policy	The system targets 99% availability with daily backups and simple recovery steps.
Auditability	StockTransaction and AuditLog-style records	Important actions such as stock deductions, case closure and payment verification can be traced.
Portability	Docker Compose deployment	The same containers run in development and production-like environments.
Accessibility	Clear labels, readable contrast and keyboard-friendly forms	UI components include labels, visible focus states and semantic form structure during implementation.

9. Design Constraints

9.1 Browser Support

Browser	Min Version	Notes
Chrome	110+	Primary development target
Firefox	110+	Full support
Safari	16+	WebSocket and CSS grid verified
Edge	110+	Chromium-based; same as Chrome
Mobile Safari / Chrome	iOS 15+ / Android 10+	Responsive layout required

9.2 Database Constraints

Constraint	Detail

Index strategy	Indexed: users.email (unique), cases.animalId, treatments.caseId, notifications.recipientId
Soft delete	Users deactivated via isActive=false; no cascade deletes to preserve case history
Session atomicity	Treatment recording + medicine stock deduction wrapped in MongoDB session to prevent partial
Supervisor immutability	cases.supervisorId set once and never updated — enforced in service layer
Collection size	No hard cap; monitored via MongoDB Atlas metrics

9.3 Hosting Assumptions

Assumption	**Detail**
VPS spec	Minimum 2 vCPU, 4 GB RAM, 40 GB SSD (Ubuntu 22.04)
Docker	Docker Engine 24+, Docker Compose v2
Concurrent users	Designed for 200 concurrent users; Redis adapter readies Socket.IO for horizontal scaling
Backup	Daily MongoDB Atlas automated backup; weekly manual export to S3
Uptime target	99% availability; Docker health checks with auto-restart
SSL	Let's Encrypt certificate via Certbot; auto-renews every 90 days
Secrets management	All secrets in .env file; never committed to Git; example in .env.example

9.4 API Constraints

Constraint	**Detail**
Pagination	All list endpoints paginated (default 20 items, max 100)

Rate limiting	100 requests/min per IP globally; 5 login attempts/15 min per IP
Payload size	Request body limit 1 MB (JSON); 5 MB multipart for file uploads
CORS	Restricted to production domain + localhost:3000 in development